

Backend Bodega

Este es el backend para el sistema de gestión de bodega, desarrollado con FastAPI y PostgreSQL.

Requisitos Previos

- Python 3.8 o superior
- Docker y Docker Compose
- PostgreSQL (se puede usar el contenedor Docker proporcionado)

Configuración del Entorno

1. Clonar el repositorio:

```
git clone [URL_DEL_REPOSITORIO]
cd backend
```

2. Crear un entorno virtual (opcional pero recomendado):

```
python -m venv venv
source venv/bin/activate # En Windows: venv\Scripts\activate
```

3. Instalar dependencias:

```
pip install -r requirements.txt
```

Configuración de la Base de Datos

El proyecto utiliza Docker Compose para la base de datos. Para iniciar la base de datos:

```
docker-compose up --build -d
```

Ejecutar la Aplicación

1. Iniciar el servidor de desarrollo:

```
uvicorn main:app --reload
```

2. La aplicación estará disponible en:

```
http://localhost:8000
```

3. Para ver la documentación de la API:

```
http://localhost:8000/docs
```

Estructura del Proyecto

- `main.py`: Punto de entrada de la aplicación y definición de endpoints
- `models.py`: Modelos de la base de datos (Proveedor, Producto, Bodega, Ubicacion, Empleado, OrdenCompra, etc.)
- `database.py`: Configuración de la base de datos
- `seed_data.py`: Script para poblar la base de datos con datos iniciales
- `requirements.txt`: Dependencias del proyecto

Modelos Principales

El sistema incluye los siguientes Modulos principales:

- Reportes
- Producto
- Bodega
- Empleado
- Inventario
- Movimiento

- [Dashboard](#)

Variables de Entorno

El proyecto utiliza las siguientes variables de entorno:

- `DATABASE_URL` : URL de conexión a la base de datos
- `POSTGRES_USER` : Usuario de PostgreSQL
- `POSTGRES_PASSWORD` : Contraseña de PostgreSQL
- `POSTGRES_DB` : Nombre de la base de datos

Despliegue con Docker

Para construir y ejecutar la aplicación usando Docker:

```
docker build -t backend-bodega .
docker-compose up
```

Notas Adicionales

- La base de datos se inicializa automáticamente al iniciar el contenedor
- Se recomienda usar el script `seed_data.py` para poblar la base de datos con datos de prueba:

```
docker-compose exec api python seed_data.py
```

- La API sigue las especificaciones de OpenAPI y está documentada en [/docs](#)
- Los endpoints principales están disponibles en la documentación de Swagger UI

Bodega FrontEnd

Este proyecto frontend fue generado con [Angular CLI](#) versión 19.0.6.

Servidor de desarrollo

Para iniciar un servidor de desarrollo local, ejecuta:

```
npm install
ng serve
```

Una vez que el servidor esté funcionando, abre tu navegador y navega a `http://localhost:4200/` . La aplicación se recargará automáticamente cada vez que modifiques cualquiera de los archivos fuente.

Generación de código

Angular CLI incluye potentes herramientas de generación de código. Para generar un nuevo componente, ejecuta:

```
ng generate component nombre-del-componente
```

Para una lista completa de esquemas disponibles (como `componentes` , `directivas` o `pipes`), ejecuta:

```
ng generate --help
```

Compilación

Para compilar el proyecto ejecuta:

```
ng build
```

Esto compilará tu proyecto y almacenará los artefactos de compilación en el directorio `dist/` . Por defecto, la compilación de producción optimiza tu aplicación para rendimiento y velocidad.

Ejecutar pruebas unitarias

Para ejecutar pruebas unitarias con el ejecutor de pruebas [Karma](#), usa el siguiente comando:

```
ng test
```

Ejecutar pruebas end-to-end

Para pruebas end-to-end (e2e), ejecuta:

```
ng e2e
```

Angular CLI no viene con un framework de pruebas end-to-end por defecto. Puedes elegir uno que se adapte a tus necesidades.

Recursos adicionales

Para más información sobre el uso de Angular CLI, incluyendo referencias detalladas de comandos, visita la página [Descripción general y referencia de comandos de Angular CLI](#).